

Explotación Manual de SQL Injection con Burp Suite sobre DVWA en Kali Linux

Este procedimiento describe una metodología paso a paso para realizar una prueba de penetración controlada sobre una aplicación vulnerable, utilizando Kali Linux, Burp Suite y DVWA (Damn Vulnerable Web Application). El objetivo es identificar y explotar manualmente una vulnerabilidad de tipo SQL Injection (SQLi), validar su impacto y documentar técnicamente los hallazgos.

Etapa 1: Preparación del Entorno y Reconocimiento

1. Iniciar DVWA

- En Kali Linux, asegúrate de tener instalado DVWA junto con Apache, MySQL y PHP.

Ejecuta:

```
sudo service apache2 start
```

```
sudo service mysql start
```

-
- Accede a la aplicación en: <http://localhost/dvwa>

2. Configurar DVWA

- Inicia sesión con las credenciales por defecto (admin/password).
- Ve a "DVWA Security" y selecciona el nivel de seguridad "Low" o "Medium".

3. Configurar Burp Suite y el Navegador

Abre Burp Suite desde Kali:

```
burpsuite
```

-
- En la pestaña **Proxy > Options**, asegúrate de que el proxy escuche en **127.0.0.1:8080**.
- En Firefox, configura manualmente el proxy en **127.0.0.1**, puerto **8080**.

Etapa 2: Intercepción y Análisis de la Solicitud

1. Acceder al Módulo Vulnerable

- URL: `http://localhost/dvwa/vulnerabilities/sqli/`

2. Interceptar la Petición

- Asegúrese de que **Intercept is on** esté activado en Burp.
- Ingresar "1" en el campo "User ID" y presionar "Submit".
- Analizar la solicitud interceptada:

GET /dvwa/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
...

- El parámetro `id` es el objetivo probable.

Etapa 3: Inyección SQL Manual y Verificación

Condición Verdadera

`id=1' AND 1=1 --`

1.
 - Reenviar la solicitud y observar la respuesta: debería mostrar información.

Condición Falsa

`id=1' AND 1=2 --`

2.
 - Si no se muestra información, se confirma la vulnerabilidad.

Etapla 4: Validación del Impacto Real

Payload para Extracción de Datos

`id=1' UNION SELECT 1, database(), 3 --`

1.
 - Si el nombre de la base de datos aparece en la respuesta, la inyección es explotable.

Etapla 5: Documentación de Resultados

- **Parámetro vulnerable:** `id`

Payloads utilizados:

`id=1' AND 1=1 --`

`id=1' AND 1=2 --`

`id=1' UNION SELECT 1, database(), 3 --`

-
- **Observaciones:**
 - Variación entre condiciones verdadera/falsa.
 - Extracción exitosa de datos usando `UNION`.
- **Riesgo estimado:** Alto

Recomendación Técnica

- Usar sentencias preparadas (prepared statements).
- Validar y sanear todas las entradas del usuario.
- Implementar control de acceso estricto y logs de actividad.
- Utilizar herramientas de escaneo automático en el ciclo de desarrollo.

Análisis de Solicitud GET Observada

GET /login.php HTTP/1.1

Host: 192.168.1.3:8080

Cache-Control: max-age=0

Accept-Language: en-US,en;q=0.9

Upgrade-Insecure-Requests: 1

User-Agent: Mozilla/5.0 (X11; Linux x86_64) Chrome/137.0.0.0

Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8

Referer: http://192.168.1.3:8080/login.php

Accept-Encoding: gzip, deflate, br

Cookie: PHPSESSID=5esj209p06gv6jamlnccbr1hl7; security=low

Connection: keep-alive

Observaciones:

- El navegador accede a **login.php** en el puerto 8080.
- El nivel de seguridad está configurado en "low" (DVWA).
- La sesión es identificada por el **PHPSESSID**.
- La cabecera **Referer** podría ser utilizada para explotar o evitar protecciones CSRF.
- La cookie y el user-agent podrían manipularse para simular otros escenarios.

Acciones Posibles:

- Inyecciones en **GET** si se usan parámetros en la URL (**login.php?user=admin' OR '1'='1**).
- Fuzzing del login para detectar errores SQL o XSS.
- Secuestro o fijación de sesión manipulando **PHPSESSID**.
- Análisis de comportamiento ante cabeceras modificadas (referer, user-agent).

